| | |
|---|---|
| BAB | : VI (6) |
| NAMA | : FAIZ IQBAL I'TISHOM |
| NIM | : 164231059 |
| TGL PRAKTIKUM | : 12 September 2025 |

## TM 6 – Signal Processing

**Lakukan seluruh percobaan pada modul ini dan berikan analisis yang kalian temukan**

### A. Import Library dan Subset Data

**Sintaks**

```
import wave
import numpy as np
import matplotlib.pyplot as plt
import os

file_path = "C:/Uner/Semester 5/Data Mining II/Coolyeah/archive/set_b"
```

```
datake = 1
file = os.listdir(file_path)[datake]
wav_file = wave.open(os.path.join(file_path,file), 'rb')
```

```
num_channels = wav_file.getnchannels()
sample_width = wav_file.getsampwidth()
frame_rate = wav_file.getframerate()  #Sample rate (samples per second)
num_frames = wav_file.getnframes()
duration = num_frames / float(frame_rate)  #dalam seconds
```

```
print(f"Channels: {num_channels}")
print(f"Sample Width: {sample_width} bytes")
print(f"Frame Rate: {frame_rate} Hz")
print(f"Number of Frames: {num_frames}")
print(f"Duration: {duration:.2f} seconds")
```

**Output**

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025

```
Channels: 1
Sample Width: 2 bytes
Frame Rate: 4000 Hz
Number of Frames: 87107
Duration: 21.78 seconds
```

**Penjelasan**

Pada tahap awal ini, dilakukan persiapan dan inspeksi data audio.

**Import Library**: Kode mengimpor beberapa pustaka Python yang esensial. wave digunakan secara spesifik untuk membaca dan memanipulasi file audio berformat .wav. numpy adalah pustaka fundamental untuk komputasi numerik, yang akan digunakan untuk mengubah data audio menjadi larik (array) matematis. matplotlib.pyplot adalah pustaka untuk visualisasi data, yang akan dipakai untuk membuat plot sinyal audio. Terakhir, os menyediakan fungsionalitas untuk berinteraksi dengan sistem operasi, seperti membaca daftar file dalam sebuah direktori.

**Pemilihan Data**: Kode menetapkan path ke direktori yang berisi dataset audio dan memilih satu file spesifik (datake = 1) untuk dianalisis sebagai sampel.

**Ekstraksi Metadata**: Dengan menggunakan pustaka wave, file audio dibuka dan metadata utamanya diekstraksi. Metadata ini krusial untuk memahami karakteristik dasar dari sinyal digital:

**Channels**: Menunjukkan jumlah kanal audio. Nilai 1 berarti audio ini adalah **mono**, di mana semua suara direkam dalam satu sinyal. Sebaliknya, audio stereo akan memiliki 2 kanal.

**Sample Width**: Menunjukkan jumlah byte yang digunakan untuk merepresentasikan setiap sampel audio. Nilai 2 bytes (atau 16-bit) menandakan resolusi atau kedalaman bit audio. Ini berarti setiap nilai amplitudo dapat direpresentasikan dalam rentang 216 (65.536) tingkatan, yang memberikan rentang dinamis yang cukup baik.

**Frame Rate (Sample Rate)**: Ini adalah frekuensi pengambilan sampel, yaitu 4000 Hz. Artinya, sinyal audio analog asli diukur sebanyak 4.000 kali per detik untuk menghasilkan sinyal digital. Menurut teorema Nyquist-Shannon, sample rate ini dapat merepresentasikan frekuensi hingga 2.000 Hz.

**Number of Frames**: Ini adalah jumlah total sampel yang ada dalam file audio.

**Duration**: Durasi total file audio dalam detik, yang dihitung dengan membagi jumlah total frame dengan frame rate (87107 / 4000 ≈ 21.78 detik).

| | |
|---|---|
| BAB | : VI (6) |
| NAMA | : FAIZ IQBAL I'TISHOM |
| NIM | : 164231059 |
| TGL PRAKTIKUM | : 12 September 2025 |

Langkah ini memberikan gambaran umum tentang properti teknis dari file audio yang akan diproses lebih lanjut.

**B. Import Library dan Subset Data**

**Sintaks**

```
frames = wav_file.readframes(num_frames)
wav_file.close()

audio_data = np.frombuffer(frames, dtype=np.int16)
```

```
if num_channels > 1:
    audio_data = np.reshape(audio_data, (-1, num_channels))
```

```
print("Audio Array:", audio_data)
print("Array Shape:", audio_data.shape)
```

```
plt.figure(figsize=(10, 4))
if num_channels > 1:
    plt.plot(audio_data[:, 0])
else:
    plt.plot(audio_data)
plt.title('Waveform of the Audio Signal')
plt.xlabel('Sample Number')
plt.ylabel('Amplitude')
plt.grid()
plt.show()
```

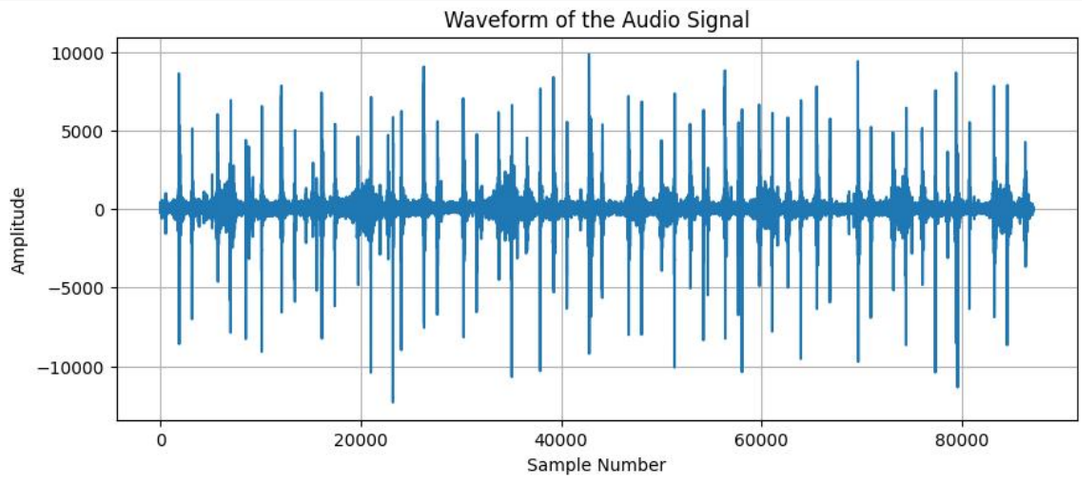
**Output**

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025




**Penjelasan**

Setelah metadata dipahami, langkah selanjutnya adalah membaca data audio itu sendiri dan memvisualisasikannya dalam domain waktu.

**Membaca Frame Audio**: Fungsi wav_file.readframes(num_frames) membaca seluruh konten data dari file audio sebagai serangkaian byte mentah (raw bytes). Setelah data dibaca, file ditutup dengan wav_file.close().

**Konversi ke Array Numerik**: Data byte mentah tidak dapat diproses secara matematis. Oleh karena itu, np.frombuffer(frames, dtype=np.int16) digunakan untuk menginterpretasikan buffer byte ini sebagai larik (array) NumPy. dtype=np.int16 ditentukan berdasarkan *Sample Width* (2 bytes = 16 bit), yang mengubah setiap 2 byte menjadi satu bilangan bulat 16-bit. Hasilnya adalah sebuah array satu dimensi yang berisi nilai amplitudo dari setiap sampel.

**Visualisasi Waveform**: Array numerik ini kemudian diplot menggunakan matplotlib.pyplot. Hasilnya adalah sebuah **waveform** atau bentuk gelombang.

**Sumbu X (Sample Number)**: Mewakili urutan sampel dari waktu ke waktu, dari awal hingga akhir audio.

**Sumbu Y (Amplitude)**: Mewakili intensitas atau "volume" sampel pada titik waktu tersebut. Nilai positif dan negatif mencerminkan fluktuasi tekanan udara dari gelombang suara. Nilai mendekati nol menunjukkan keheningan, sedangkan nilai yang jauh dari nol (puncak tinggi dan lembah dalam) menunjukkan suara yang keras.

Grafik ini adalah representasi sinyal audio dalam **domain waktu**, yang secara visual menunjukkan bagaimana amplitudo suara berubah sepanjang durasi file.

**C. Visualisasi Satu Sampel Data**

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025

**Sintaks**

```
from IPython.display import Audio

audio_to_play = audio_data[:,0] if num_channels > 1 else audio_data



Audio(audio_to_play, rate=frame_rate)
```

**Output**

**Penjelasan**

Langkah ini bertujuan untuk melakukan verifikasi auditori atau mendengarkan langsung sampel data yang telah dibaca dan diproses.

**Import Library Audio**: Kode mengimpor kelas Audio dari IPython.display, sebuah fitur yang tersedia di lingkungan seperti Jupyter Notebook atau Google Colab.

**Membuat Objek Audio**: Objek Audio dibuat dengan memberikan dua argumen utama:

- audio_to_play: Array NumPy yang berisi data amplitudo sinyal.

- frame rate: Frame rate atau laju sampel dari audio (4000 Hz). Parameter ini sangat penting; tanpanya, audio akan diputar dengan kecepatan yang salah, menyebabkan distorsi pada nada (pitch).

Output yang dihasilkan adalah sebuah pemutar audio interaktif. Langkah ini berfungsi sebagai:

- *sanity check* untuk memastikan bahwa data telah dibaca dan diinterpretasikan dengan benar dari file .wav ke dalam array NumPy, dan bahwa suara yang didengar sesuai dengan yang diharapkan.

**D. Visualisasi Satu Sampel Data**

**Sintaks**

```
from scipy.signal import spectrogram



f,t, sxx = spectrogram(audio_data[:,0] if num_channels > 1 else audio_data, frame_rate)

plt.pcolormesh(t,f,10 * np.log10(sxx))
```

BAB : VI (6)

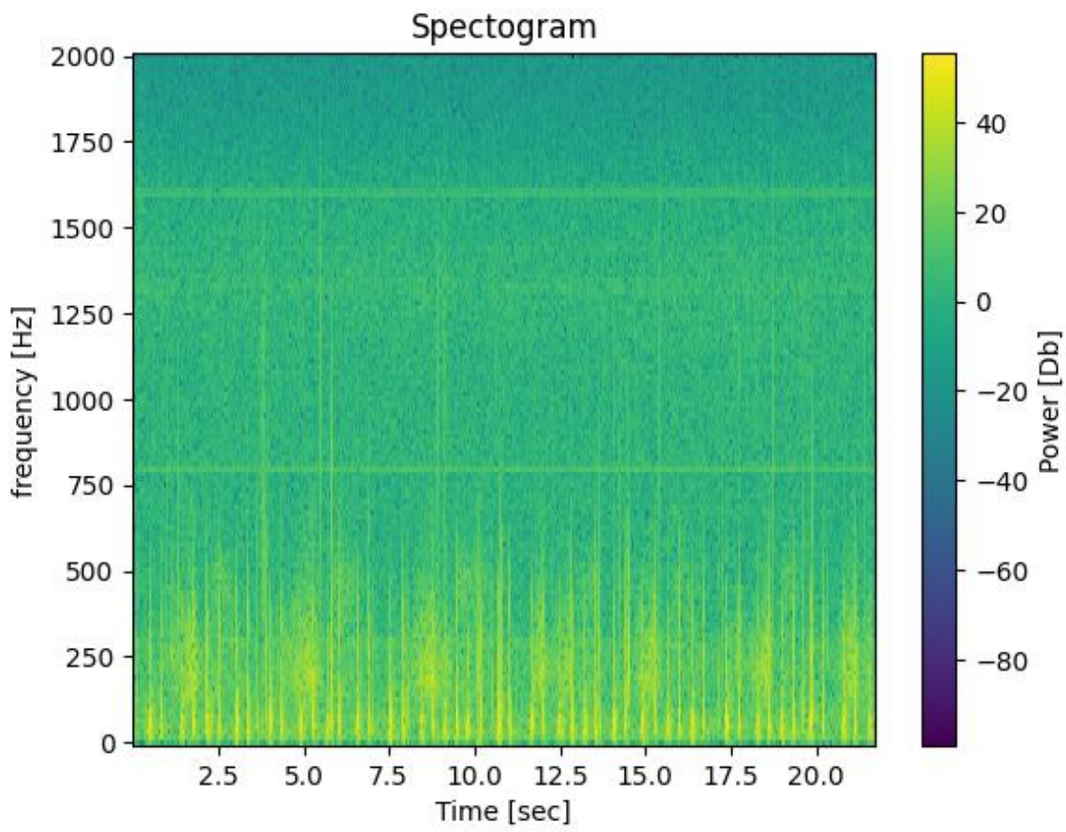
NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025

```
plt.ylabel('frequency [Hz]')
plt.xlabel("Time [sec]")
plt.title("Spectogram")
plt.colorbar(label = 'Power [Db]')
plt.show()
```

**Output**




**Penjelasan**

Visualisasi waveform hanya menunjukkan perubahan amplitudo terhadap waktu. Untuk analisis yang lebih mendalam, kita perlu melihat bagaimana konten frekuensi sinyal berubah seiring waktu. Untuk tujuan ini, **spektogram** digunakan.

**Perhitungan Spektogram**: Fungsi spectrogram dari pustaka scipy.signal digunakan untuk menghitung *Short-Time Fourier Transform (STFT)* dari sinyal audio. STFT bekerja dengan memecah sinyal menjadi segmen-segmen waktu yang pendek dan tumpang tindih, lalu menerapkan Transformasi Fourier pada setiap segmen untuk menentukan spektrum frekuensi di dalamnya.

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025

**Visualisasi**: Hasilnya diplot menggunakan plt.pcolormesh.

**Sumbu X (Time [sec])**: Menunjukkan durasi audio dalam detik.

**Sumbu Y (frequency [Hz])**: Menunjukkan rentang frekuensi yang ada dalam sinyal.

**Sumbu Warna (Power [Db])**: Intensitas warna pada setiap titik (waktu, frekuensi) merepresentasikan kekuatan atau energi (power) dari frekuensi tersebut pada waktu tertentu. Skala warna diukur dalam desibel (dB), yang merupakan skala logaritmik (10 * np.log10(sxx)), sehingga lebih sesuai dengan persepsi pendengaran manusia. Warna yang lebih cerah (kuning) menunjukkan frekuensi dengan energi yang lebih tinggi (lebih dominan), sedangkan warna yang lebih gelap menunjukkan energi yang lebih rendah.

Spektogram adalah visualisasi dalam **domain waktu-frekuensi** yang jauh lebih informatif daripada waveform, karena ia mengungkapkan komponen frekuensi mana yang aktif pada setiap momen dalam sinyal audio.

**Analisis Clustering Signal Processing**

Bagian ini berfokus pada pengelompokan (clustering) seluruh file audio dalam dataset berdasarkan karakteristik suaranya. Tujuannya adalah untuk secara otomatis menemukan grup-grup audio yang memiliki kemiripan satu sama lain.

**1. Ekstraksi Fitur dari Semua File Audio**

**Sintaks**

```
import os
import numpy as np
import librosa
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import pandas as pd

file_path = "C:/Uner/Semester 5/Data Mining II/Coolyeah/archive/set_b"

all_features = []
```

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025

```
all_filenames = []
```

```
print("Memulai ekstraksi fitur dari file audio...")
```

```
for filename in os.listdir(file_path):
    if filename.endswith('.wav'):
        full_path = os.path.join(file_path, filename)

        try:
            y, sr = librosa.load(full_path, sr=None)
            mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)
            mfccs_mean = np.mean(mfccs.T, axis=0)

            all_features.append(mfccs_mean)
            all_filenames.append(filename)
        except Exception as e:
            print(f"Error memproses file {filename}: {e}")
```

```
print(f"Ekstraksi fitur selesai. Total {len(all_features)} file diproses.")

X = np.array(all_features)
```

**Output**

```
Memulai ekstraksi fitur dari file audio...
Ekstraksi fitur selesai. Total 656 file diproses.
```

**Penjelasan**

Machine learning seperti clustering tidak dapat bekerja langsung pada data audio mentah. Oleh karena itu, kita perlu mengekstrak fitur-fitur numerik yang representatif dari setiap file audio.

**Iterasi File**: Kode ini mengotomatiskan proses dengan melakukan iterasi pada setiap file yang berakhiran .wav di dalam direktori yang ditentukan.

**Ekstraksi Fitur MFCC**: Untuk setiap file, fitur yang diekstrak adalah **Mel-**

| | |
|---|---|
| BAB | : VI (6) |
| NAMA | : FAIZ IQBAL I'TISHOM |
| NIM | : 164231059 |
| TGL PRAKTIKUM | : 12 September 2025 |

**Frequency Cepstral Coefficients (MFCCs)** menggunakan pustaka librosa. MFCC adalah fitur standar emas dalam pemrosesan audio dan ucapan karena kemampuannya meniru persepsi pendengaran manusia. Prosesnya secara singkat adalah:

Sinyal dianalisis spektrumnya.

Spektrum frekuensi tersebut dipetakan ke **skala Mel**, yang lebih sensitif terhadap perubahan frekuensi rendah (seperti telinga manusia).

Hasilnya kemudian dianalisis secara *cepstral* untuk mendapatkan koefisien yang ringkas dan informatif.

n_mfcc=20 berarti kita mengekstrak 20 koefisien untuk setiap frame audio.

**Agregasi Fitur**: Karena MFCC menghasilkan matriks fitur (20 koefisien untuk banyak frame waktu), kita memerlukan satu vektor fitur tunggal untuk setiap file. Ini dicapai dengan menghitung nilai rata-rata (np.mean) dari setiap koefisien di seluruh durasi file. Hasilnya adalah satu vektor berukuran 20 untuk setiap file audio.

**Penyusunan Dataset**: Vektor-vektor fitur ini dikumpulkan dalam sebuah list (all_features) yang kemudian diubah menjadi sebuah array NumPy besar bernama X.

Output Total 656 file diproses menunjukkan bahwa proses ini berhasil mengekstrak fitur dari 656 file audio, menciptakan dataset untuk langkah selanjutnya.

## 2. Standardisasi Fitur

**Sintaks**

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


print("\nBentuk data fitur setelah scaling:", X_scaled.shape)
```

**Output**

```
Bentuk data fitur setelah scaling: (656, 20)
```

**Penjelasan**

Algoritma clustering seperti K-Means sangat sensitif terhadap skala fitur. Jika satu fitur memiliki rentang nilai yang jauh lebih besar dari yang lain, fitur tersebut akan mendominasi perhitungan jarak dan menghasilkan clustering yang tidak akurat.

| | |
|---|---|
| BAB | : VI (6) |
| NAMA | : FAIZ IQBAL I'TISHOM |
| NIM | : 164231059 |
| TGL PRAKTIKUM | : 12 September 2025 |

**Inisialisasi StandardScaler**: Objek StandardScaler dari scikit-learn dibuat.

**Proses Scaling**: Metode fit_transform(X) diterapkan pada data fitur X. Metode ini melakukan dua hal:

**Fit**: Menghitung rata-rata (mean) dan simpangan baku (standard deviation) untuk setiap kolom (setiap fitur MFCC) dari data.

**Transform**: Mengubah setiap nilai dalam data dengan menguranginya dengan rata-rata kolomnya dan membaginya dengan simpangan baku kolomnya (juga dikenal sebagai normalisasi Z-score).

**Hasil**: Setelah standardisasi, setiap fitur dalam dataset X_scaled akan memiliki **rata-rata 0** dan **simpangan baku 1**. Ini memastikan bahwa semua 20 fitur memiliki skala yang sebanding dan akan memberikan kontribusi yang setara pada proses clustering.

(656, 20) mengonfirmasi bahwa kita memiliki 656 sampel (file audio), masing-masing direpresentasikan oleh 20 fitur yang telah distandarisasi.

### 3. Clustering dengan K-means

**Sintaks**

```
num_clusters = 3


kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)


results_df = pd.DataFrame({
    'filename': all_filenames,
    'cluster': clusters
})
results_df = results_df.sort_values(by='cluster')


print("\nHasil Clustering:")
print(results_df)


print("\nDistribusi file per cluster:")
print(results_df['cluster'].value_counts())
```

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025

**Output**

```
Hasil Clustering:
                                        filename  cluster
653            normal__298_1311685888900_B.wav        0
649           normal__294_1311681084248_D1.wav        0
19    Bunlabelledtest__115_1306259437619_D.wav        0
20    Bunlabelledtest__116_1306258689913_B.wav        0
22    Bunlabelledtest__118_1306262335509_A.wav        0
..                                         ...      ...
28    Bunlabelledtest__123_1306331925797_A.wav        2
25    Bunlabelledtest__121_1306263877235_B.wav        2
24   Bunlabelledtest__118_1306262335509_C2.wav        2
21   Bunlabelledtest__117_1306262456650_B1.wav        2
16    Bunlabelledtest__110_1305655332337_D.wav        2

[656 rows x 2 columns]

Distribusi file per cluster:
cluster
1    280
0    229
2    147
Name: count, dtype: int64
```

**Penjelasan**

Dengan data fitur yang telah siap, sekarang kita dapat menerapkan algoritma clustering K-Means.

**Inisialisasi K-Means**: Model KMeans diinisialisasi dengan beberapa parameter penting:

- n_clusters=3: Menentukan bahwa kita ingin mengelompokkan 656 file audio ke dalam 3 cluster yang berbeda.

- random_state=42: Menjamin bahwa hasil clustering akan selalu sama setiap kali kode dijalankan (reproducibility).

- n_init=10: Mengatasi sensitivitas K-Means terhadap posisi awal centroid. Algoritma akan dijalankan 10 kali dengan posisi awal yang berbeda, dan hasil terbaik (dengan inersia terendah) akan dipilih.

**Pelatihan dan Prediksi**: Metode kmeans.fit_predict(X_scaled) melatih model pada data yang telah diskalakan dan langsung memberikan label cluster (0, 1, atau 2) untuk setiap file audio.

| | |
|---|---|
| BAB | : VI (6) |
| NAMA | : FAIZ IQBAL I'TISHOM |
| NIM | : 164231059 |
| TGL PRAKTIKUM | : 12 September 2025 |

**Analisis Hasil**: Hasilnya disajikan dalam DataFrame pandas untuk kemudahan interpretasi, menunjukkan nama file dan cluster tempatnya berada. Perhitungan value_counts() menunjukkan distribusi file di setiap cluster: Cluster 1 adalah yang terbesar dengan 280 file, diikuti oleh Cluster 0 dengan 229 file, dan Cluster 2 dengan 147 file.

Langkah ini berhasil mengelompokkan ratusan file audio ke dalam tiga kategori berbeda berdasarkan kemiripan fitur MFCC mereka.

### 4. Visualisasi Hasil dengan PCA

**Sintaks**

```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)



plt.figure(figsize=(12, 8))

scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis', alpha=0.7)
```

```
plt.title('Visualisasi Cluster Audio dengan PCA', fontsize=16)

plt.xlabel('Principal Component 1')

plt.ylabel('Principal Component 2')

plt.grid(True)

plt.legend(handles=scatter.legend_elements()[0], labels=[f'Cluster {i}' for i in range(num_clusters)])

plt.show()
```

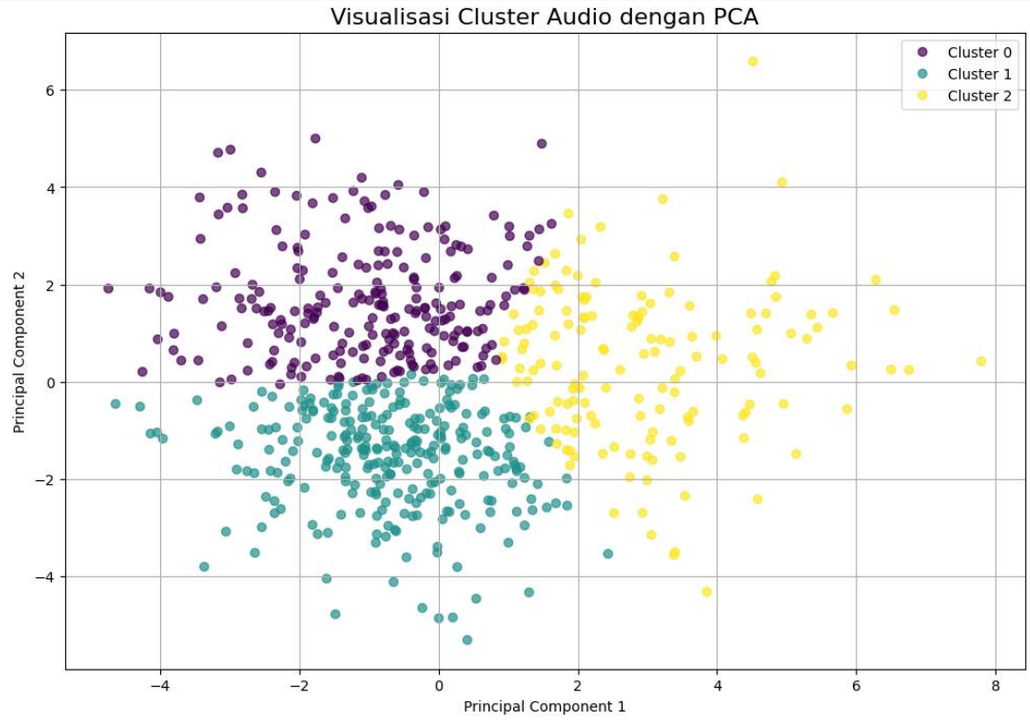
**Output**

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025




**Penjelasan**

Data fitur asli memiliki 20 dimensi, yang tidak mungkin divisualisasikan secara langsung. Untuk memeriksa hasil clustering secara visual, kita perlu mengurangi dimensinya.

**Reduksi Dimensi dengan PCA**: **Principal Component Analysis (PCA)** adalah teknik reduksi dimensi yang digunakan untuk mengubah data berdimensi tinggi menjadi data berdimensi lebih rendah sambil mempertahankan sebanyak mungkin variasi dari data asli.

− PCA(n_components=2) diinisialisasi untuk mereduksi 20 dimensi fitur menjadi hanya 2 dimensi (dua komponen utama).

−pca.fit_transform(X_scaled) menghitung dua komponen utama tersebut dan memproyeksikan data ke ruang 2D yang baru ini.

**Visualisasi Scatter Plot**: Data 2D yang dihasilkan kemudian diplot sebagai diagram pencar (*scatter plot*).

- Setiap titik pada plot mewakili satu file audio.

- Posisi titik pada sumbu X dan Y ditentukan oleh nilainya pada *Principal Component*

BAB : VI (6)

NAMA : FAIZ IQBAL I'TISHOM

NIM : 164231059

TGL PRAKTIKUM : 12 September 2025

*1* dan *Principal Component 2*.

- Warna setiap titik (c=clusters) ditentukan oleh label cluster yang diberikan oleh K-Means.

**Interpretasi Plot**: Plot ini memungkinkan kita untuk menilai secara kualitatif seberapa baik clustering bekerja. Idealnya, titik-titik dengan warna yang sama akan berkumpul bersama dan terpisah dari warna lain. Dalam plot ini, kita dapat melihat bahwa Cluster 2 (kuning) cenderung membentuk kelompok yang cukup berbeda. Sementara itu, Cluster 0 (ungu) dan Cluster 1 (hijau kebiruan) tampak lebih tersebar dan memiliki beberapa tumpang tindih, menunjukkan bahwa perbedaan sonik antara kedua cluster ini mungkin tidak sekontras perbedaan mereka dengan Cluster 2.

Visualisasi ini memberikan konfirmasi visual yang intuitif bahwa algoritma K-Means telah berhasil menemukan struktur pengelompokan yang ada di dalam data audio.